

拟定规范：下一代意图驱动型智能软件过程与管理规范 (V 2026.1.0)

第一章 总则与理念框架

1.1 目的与适用范围

本规范文档旨在定义基于大语言模型（LLM）与多智能体（Multi-Agent）技术重构后的全新软件开发生命周期（SDLC）。传统的“瀑布模型”或“敏捷迭代”在面对 2026 年的高度智能化开发工具时，其原有的物理角色隔离、线性文档传递机制已成为制约效能的瓶颈。

本规范建立了一套以“意图驱动（Intent-Driven）”、“人机协同（Human-Machine Collaboration）”和“自愈弹性（Self-Healing）”为核心的新型软件过程规范。适用于所有采用智能化工具链（如 Cursor, RAG 知识库, 自动化评测 Agent）进行复杂系统构建的研发团队。

1.2 核心指导原则

- 意图高于语法：**开发的核心活动从“编写代码”转移到“准确表达业务意图与系统约束”。代码不再是人类直接生产的第一资产，而是由智能体根据意图渲染的中间产物。
- 极速微循环反馈：**摒弃传统长达数周的 Sprint 周期。人机结对开发要求在数分钟乃至数秒内完成“意图输入 -> 代码生成 -> AST 静态审查 -> 自我修正 -> 人类接管”的微循环。
- 评估前置与内建质量（Quality Built-in）：**测试不再是开发的后置环节。系统约束和评估标准必须在需求捕获阶段即被形式化定义，并转化为驱动 Agent 编码的“护栏”。

1.3 核心术语与定义

- SDLC (Software Development Life Cycle):** 软件开发生命周期，在本规范中特指被 AI 重构后的全链路智能研发过程。
- Intent (意图):** 业务需求、架构约束和非功能性需求的总和。以自然语言或高度抽象的 DSL（领域特定语言）表达。
- Agent (智能体):** 具备感知上下文、自主规划任务、执行动作并具备一定自我反思（Reflection）能力的 AI 实体。
- RAG (Retrieval-Augmented Generation):** 检索增强生成。在本过程中，用于将企业历史架构文档、API 资产动态注入大模型上下文的知识库技术。

第二章 组织架构与协同角色规范

在新型智能软件过程中，传统的“产品经理-前端-后端-测试”的流水线分工被打破，团队被重塑为高度自治的“超级个体”与“数字员工（智能体）”的混合编队。

2.1 人类核心角色规范

2.1.1 业务意图架构师 (Business Intent Architect, BIA)

- 角色定位：** 系统的“大脑”与业务价值的定义者，由传统的产品经理与系统架构师融合而来。
- 核心职责：**
 - 负责通过自然语言结合企业领域知识库，向多智能体系统输入宏观业务意图与非功能性约束。
 - 负责审查和微调 AI 生成的系统级架构图、API 契约和数据库模型。
 - 负责跨业务领域的逻辑串联，解决 AI 在多模块协同中产生的逻辑断层。
- 能力要求：** 领域驱动设计能力 (DDD)、高阶 Prompt 工程能力、业务价值流拆解能力。

2.1.2 AI 协同开发专家 (AI Orchestration Developer, AOD)

- 角色定位：** 代码质量的最终“守门员”与 Agent 指挥官。
- 核心职责：**
 - 在智能 IDE（如 Cursor）中拆解宏观意图，为底层 Coder Agent 分配具体的上下文视界（Context Window）。
 - 执行高密度的代码审查（Code Review），拦截并修正 AI 产生的“幻觉”代码。
 - 处理高复杂度的技术债务、底层算法重构及跨模块依赖升级等当前 AI 难以独立闭环的深水区任务。
- 能力要求：** 模式识别能力、底层源码级别的 Debug 能力、多范式编程框架的深刻理解。

2.1.3 智能治理与质控分析师 (AI Governance & Quality Analyst, AQA)

- 角色定位：** 系统护栏的设计者，替代了传统的“点点点”测试工程师。
- 核心职责：**
 - 设计自动化评估框架（Evaluation Frameworks），定义验收标准并将其转化为可被 Test Agent 读取的约束条件。
 - 配置和监控视觉自愈测试流水线，确保 UI 变更不会导致测试崩溃。
 - 监控模型在生产环境中的数据安全合规性及性能退化情况。

2.2 数字员工（机器智能体）阵列定义

- 产品与架构 Agent：** 驻留在需求平台，负责将自然语言转化为 Mermaid UML 图、API Swagger 规范和 ER 模型。
- 编码自治 Agent：** 驻留在开发者 IDE 内部，具备全代码库扫描、依赖树分析和多文件协同生成的执行力。
- 自愈测试 Agent：** 驻留在 CI/CD 流水线，具备基于视觉识别和 DOM 语义分析的动态寻路与测试脚本自我修复能力。

第三章 全生命周期流程规范 (全景图与详述)

本章节是新版软件过程管理的核心，详细规定了从“意图产生”到“价值交付”的标准操作程序（SOP）。

3.1 过程全景总览视图

第一阶段: 动态意图捕获与

业务意图架构师 BIA

Unsupported markdown:
list



对抗辩论与逻辑消除

审批/微调



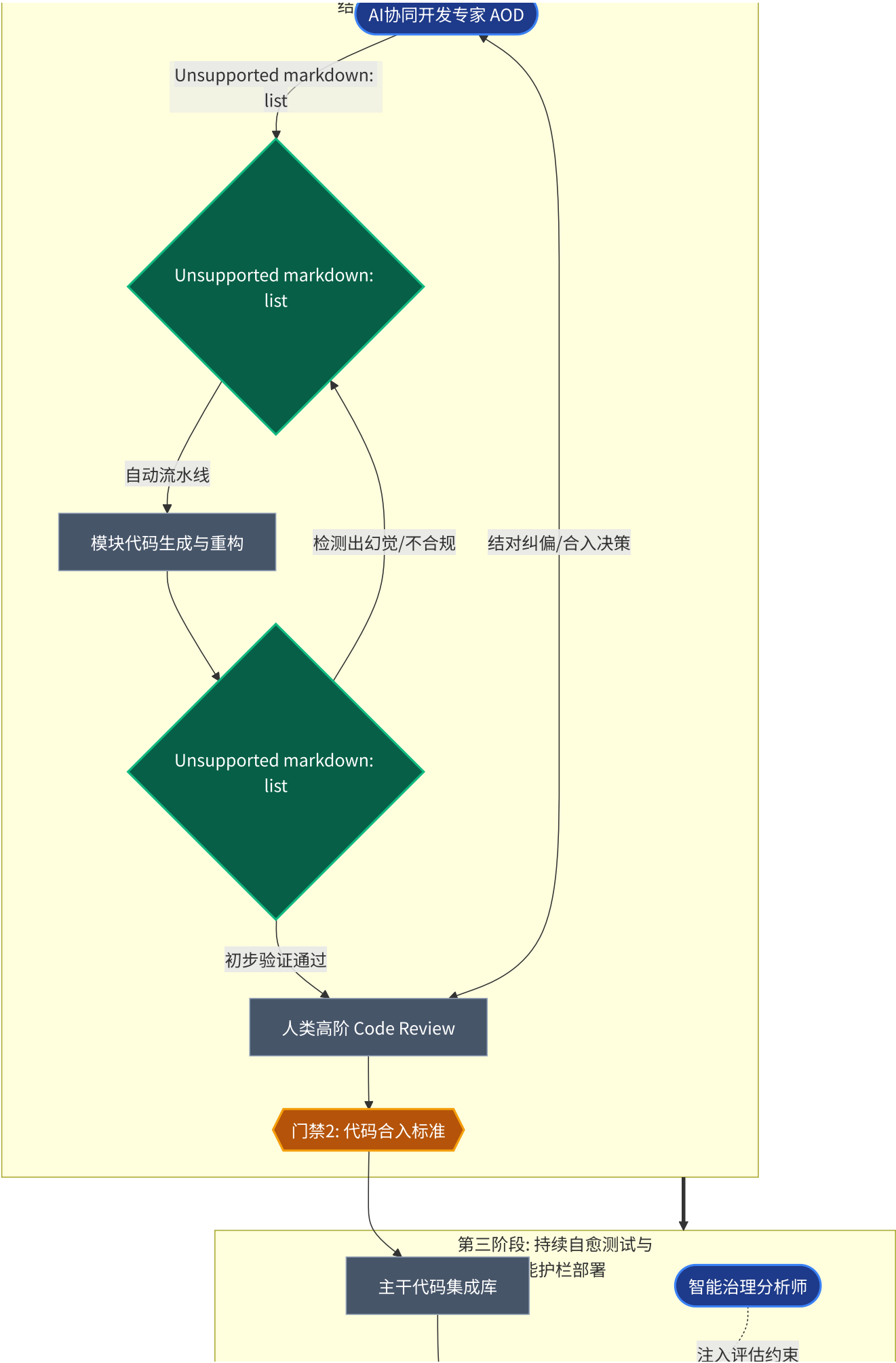
零损耗转换

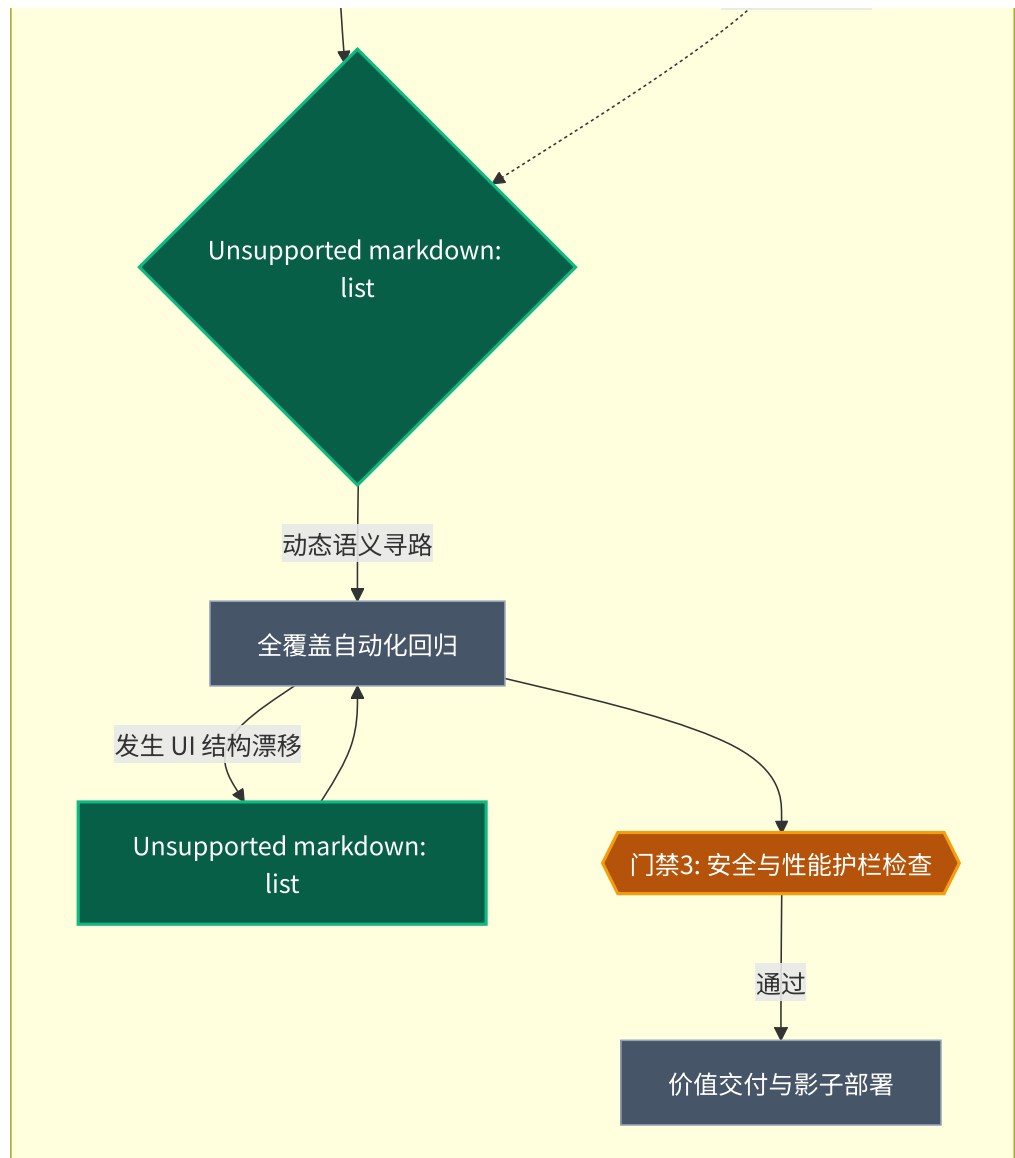


门禁1: 架构意图评审

第二阶段: 自治编码与人机







3.2 第一阶段：动态意图捕获与智能建模规范

本阶段的目的是将模糊的人类业务需求，转化为严格、无二义性且机器可读的架构设计资产。彻底废除传统的 Word 版 PRD 文档。

1. 输入准则 (Inputs):

- 业务战略目标（OKR）。
- 自然语言形式的用户故事（User Stories）或验收标准草案。
- 企业现有的 RAG 知识库（包含历史代码架构规范、三方 API 密钥规范）。

2. 标准操作程序 (SOP):

- **Step 1: 意图注入。** BIA 登录智能需求平台，通过多轮对话形式，将核心业务流程描述给系统。
- **Step 2: 知识库对齐。** 平台大模型自动检索 RAG 库，分析新意图是否与现有系统中的数据库表结构、核心支付链路等发生冲突，若有冲突，立即高亮提示 BIA。
- **Step 3: 多智能体辩论 (Multi-Agent Debate)。** 产品分析 Agent 生成功能链路，而架构推演 Agent 负责从高可用、高并发角度进行“找茬”。两者的日志对 BIA 完全透明。

- **Step 4: 资产渲染。**达成一致后，AI 瞬间并行生成以下标准格式资产：Mermaid 时序图、OpenAPI 3.0 契约文档、Prisma/SQL 数据库模型描述。

3. 输出物 (Outputs):

- 形式化、可直接导入 IDE 的系统架构说明书。

4. 质量门禁 1 (Gate 1 - 架构意图评审):

- 由 BIA 进行最终审批。核对 API 接口边界是否清晰、非功能性需求（如 TPS>10000）是否在设计中体现。未通过则打回 Step 1 重新调整 Prompt。

3.3 第二阶段：自治编码与人机结对微循环规范

本阶段是研发效能实现 10 倍速跃升的核心区域。AOD 不再从零编写“胶水代码”，而是作为指挥官引导 AI 生成并进行防线构筑。

1. 输入准则 (Inputs):

- 通过门禁 1 的 OpenAPI 契约和 UML 架构资产。
- 项目的全局上下文（Codebase Context）。

2. 标准操作程序 (SOP):

- **Step 1: 上下文圈定。**AOD 在 Cursor IDE 中，根据当前任务拆分，通过 `@Files` 或 `@Folders` 语法，为 Coder Agent 精准投喂必要的上下文范围，防止大模型陷入“信息过载”。
- **Step 2: 指令生成。**AOD 发出明确的业务实现指令（例如：“参考既有的 UserAuth 中间件逻辑，实现刚生成的 /payment/checkout 接口，并确保调用了分布式锁”）。
- **Step 3: 自治编码与本地修正。**Agent 开始多文件同步生成。生成过程中，本地的静态语法树（AST）审查工具若发现空指针、未处理异常，将自动触发 Agent 进行“反思（Reflection）”并局部重写，此过程无需人类介入。
- **Step 4: 结对审查。**Agent 生成完毕并确保编译通过后，向 AOD 提交 Diff（代码差异视图）。AOD 重点审查业务逻辑的深层漏洞、设计模式的正确性以及潜在的安全注入风险。

3. 输出物 (Outputs):

- 具备高度可读性、符合企业编码规范的业务代码模块，以及同行评审记录。

4. 质量门禁 2 (Gate 2 - 代码合入标准):

- 代码必须通过本地的自动气味检测（Code Smell）、圈复杂度检测低于 15。
- 必须通过 AOD 的人工 Code Review 并电子签名。
- 生成的代码必须附带对应的 AI 生成日志摘要（证明非抄袭且符合合规性）。

3.4 第三阶段：持续自愈测试与智能护栏部署规范

本阶段废除传统的“写完代码再写测试用例”的滞后模式。测试资产在代码生成的同构瞬间被自动生成，并具备面对 UI 变化的弹性韧性。

1. 输入准则 (Inputs):

- 通过门禁 2 并合并入主干的分支代码。

- AQA 提前注入的评估驱动框架（EDD）指标。

2. 标准操作程序 (SOP):

- Step 1: AST 逆向单元测试生成。** 代码合入瞬间，后台 CodiumAI 等工具通过读取 AST 语法树结构，自动逆向推导并生成 100% 涵盖所有 if-else 分支、边界条件、异常抛出路径的单元测试用例集，并自动执行。
- Step 2: 视觉语义 UI 回归测试。** Test Agent 启动端到端（E2E）测试。不依赖脆弱的 XPath，而是模拟人类视觉，寻找带有“支付”、“确认”等语义的页面元素执行点击与表单输入。
- Step 3: 触发自愈机制。** 若前端 AOD 对页面结构进行了重构（如 `div` 改为了 `section`），Test Agent 报错捕获异常后，会在 2 秒内调取大语言模型进行页面重新理解，自动寻找到迁移后的目标元素，并持久化更新测试脚本库（Self-Healing），然后继续执行后续测试。
- Step 4: 护栏与影子部署。** AQA 监控安全护栏检测报告（如是否有脱敏数据泄露）。一切就绪后，系统自动部署到影子环境接收生产环境的真实镜像流量进行最终稳定性验证。

3. 输出物 (Outputs):

- 全覆盖的自动化测试报告、自愈日志审计记录、具备生产就绪状态的可执行容器镜像。

4. 质量门禁 3 (Gate 3 - 安全与性能护栏检查):

- 单元测试行覆盖率必须 > 90%。
- 安全扫描 Agent 未报告任何 高/危 级别漏洞。
- 所有自愈测试用例闭环率达到 100%。

第四章 核心过程支撑与管理规程

为了确保智能软件生命周期不因过度依赖 AI 而陷入混乱，必须建立强大的底层支撑过程。

4.1 知识与意图资产配置管理

在智能 SDLC 中，代码只是意图的派生物。因此，版本控制的核心从“代码版本控制”升级为“意图与 Prompt 资产版本控制”。

- Prompt as Code (提示词即代码):** 所有输入给模型的核心系统级指令、企业级架构约束（Rules for AI），必须作为 `.md` 或 `.json` 格式存储在 Git 仓库中，接受与代码同等严格的版本管理。
- 上下文快照体系:** 任何一次大规模的 AI 自动化重构，系统必须自动留存当前的“上下文投喂快照（Context Snapshot）”，以便在出现“模型幻觉”导致系统灾难时，能够精确回溯是哪些知识库内容的检索污染了生成结果。

4.2 智能合规与模型幻觉治理机制 (Guardrails)

- 数据安全红线屏蔽（Data Anonymization Pipeline）:**
禁止开发人员将包含用户真实 PII（个人敏感信息）、企业核心定价算法、生产数据库密钥的代码片段投喂给外部公有云大模型（如 Claude 3.5）。所有离开企业内网的上下文请求，必须经过网关层的“数据清洗 Agent”进行拦截和脱敏替换（Tokenization）。
- 幻觉交叉验证规程（Cross-Validation）:**
对于涉及资金交易、权限控制等 P0 级别核心模块的生成代码，必须强制启用“双模型交叉验证”：例如使用 GPT-4o 生成逻辑，立刻流转到本地化部署的 Llama 3 大模型进行恶意行为扫描与逻辑正确性二次证明。

3. 能力退化防范措施 (The "White-Box Day" Policy):

为防止人类开发人员在长期的人机协同中丧失对系统底层的掌控力，团队规范设定每月的最后一个周五为“白盒日”。当日禁止使用任何 Agent 代码生成功能，所有核心组件级别的 Bug 必须由 AOD 手动梳理抽象语法树和内存转储文件 (Dump) 进行修复。以此保证人类工程师的硬核工程兜底能力。

第五章 过程度量与持续改进 (Process Metrics)

新型过程规范下，度量指标必须从传统的工时统计转向“人机协作效率”与“系统自治程度”。团队需按双周为周期采集以下指标并进行过程回顾：

5.1 核心智能化效能度量指标

- AI 代码贡献率 (AI-Authored Ratio, AAR):** 最终合入主干的代码中，未经人类修改直接由 Agent 生成的字符比例。目标基线：> 75%。
- 微循环介入成本 (Micro-Cycle Intervention Rate, MIR):** AOD 在引导代码生成时，平均每生成 1000 行代码所需输入的 Prompt 修正次数。该指标越低，说明前期 BIA 的意图传递越精准，或者上下文知识库质量越高。
- 自愈闭环成功率 (Self-Healing Success Rate, SSR):** 当页面发生 UI 变更时，Test Agent 成功自主修复测试脚本而不需人类手动修正的比例。目标基线：> 85%。
- 意图到交付前置时间 (Intent-to-Delivery Lead Time):** 从 BIA 提交首个架构意图开始，到代码通过所有门禁部署至影子环境的总耗时。2026 年标准应以“小时”计，而非传统的“天/周”。

5.2 持续改进机制 (Retrospective)

根据上述指标度量，若某模块的 MIR 过高（开发反复纠正 AI），团队不得归咎于“模型太笨”，而应立即启动 Root Cause Analysis (根因分析)。常规改进动作包括：

- 更新或清洗企业 RAG 知识库中过时、矛盾的旧架构文档。
- 将频繁出错的特定业务规则固化为新的 `.cursorrules` 强制约束文件。
- 安排 BIA 重新进行领域驱动设计的培训，优化意图表达的颗粒度。